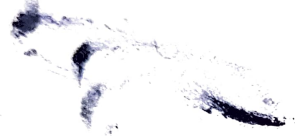


→ Complexity classes

Absent
note



Problems

Solvable

unsolvable

Tractable

Intractable

Algorithm complexity
is poly

eg:-

Path problem

$O(n)$

eg:- ① knapsack problem

② Traveling salesman

→ Algorithm complexity
is exponential

→ faster growth

Time comp = $O(n^2 2^n)$
problem

complexity classes are:-

① class P

② class NP

③ class NP-hard

④ class NP-complete.

Class P

→ polynomial time

eg:- path algorithm

Time complexity $= O(n^k)$
 $n \rightarrow$ size of input
 $k \rightarrow$ constant

eg:- path problem :- (tractable)

→ Algorithm:-

Given directed graph G, determine whether a directed path exists from s to t.

Time complexity $= O(n)$

→ Input $\langle G, s, t \rangle$

$n \rightarrow$ total no. of vertices

$G \rightarrow$ Graph directed
s & t are the ~~2 nodes~~ nodes/vertex (nodes)

① place a mark on node 's' & enqueue it to an empty queue.

② Repeat step ① until queue is empty.

③ Dequeue front element 'a' mark all unvisited neighbours of 'a' & enqueue those into Queue.

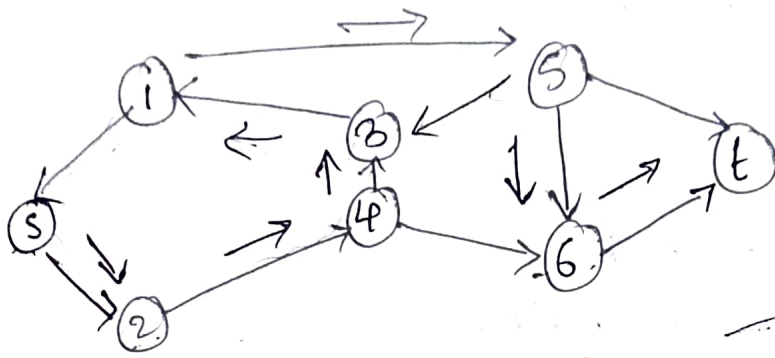
if 't' is marked, then accept. otherwise Reject
→ single source shortest path using Dijkstra's algorithm
ex:- min cost spanning tree using Prim's or Kruskal's method.

→ ② class NP

→ Note:- solved in exponential time & verified in polynomial time or factorial time

eg:- Hamilton/Ham path.

→ It is a directed path goes through each node exactly one



ex:-

→ Hamiltonian Verifier Algorithm:-

→ hamiltonian path connecting 2 specified nodes
→ no poly soln

Input: $\langle G, s, t, P \rangle$ (path is the P) $m \rightarrow$ no. of nodes in P

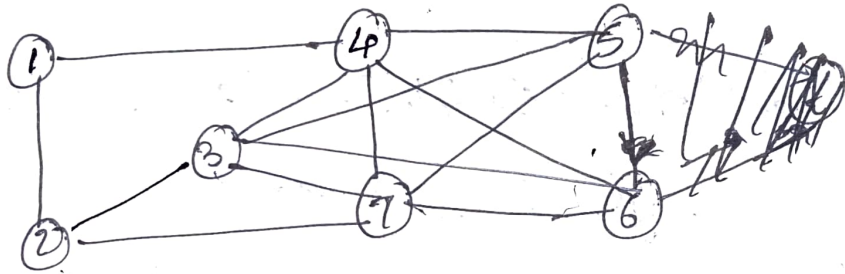
- ① check $s = p_1$ & $t = p_m$ either fails then rejects it
- ② check for the repetitions of the nodes in the least P . if any are found then reject it.
- ③ for each ' i ', check whether (p_i, p_{i+1}) is an edge in G . Here i varies from 1 to $m-1$, if any are not, reject
- ④ if all test have passed, then accept it.

→ This verifier alg run in poly time.

→ $\therefore$ Hamiltonian problem is a NP problem.

class NP
efficient problem

* → clique is an undirected problem is a sub-graph where every two nodes are connected by an edge.



→ To determine whether a graph contains a clique of specified size.

→ No polynomial time algorithm to find out the clique (but it can verify in polynomial time)

→ clique verification algorithm

$$C = \{v_1, v_2, \dots, v_k\}$$

$k=2$

Input $\langle G, k, V' \rangle$ $k \rightarrow$ clique size V' - vertex subset of graph

① Test whether V' is the set of k vertices in the graph G .

② check whether for each pair

$(u, v) \in V'$: the edge $(u, v) \in E$

Step pass, then accept, otherwise

② Both

Reject

class NP

→ complexity is polynomial time

→ exist poly time verifier
if it has solved

③ polynomial time Reduction

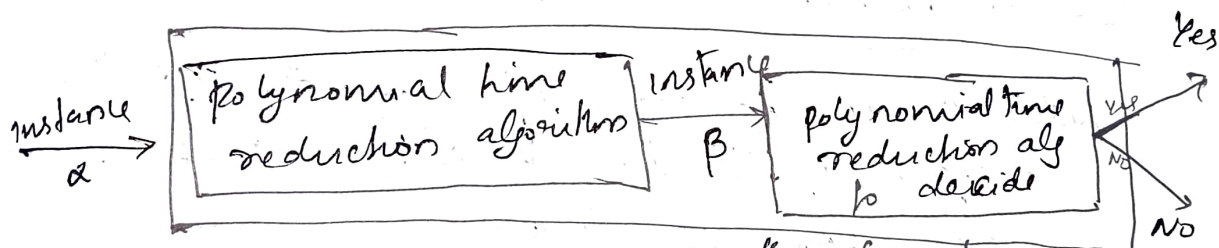
→ suppose we have 2 decision problems a & b.

→ problem a: like to solve in polynomial time
is not exist

→ instance ^A is α (alpha).

→ problem b: - having a polynomial time algorithm

→ instance is β (beta)



→ The transformation takes poly time

→ the ans are the same
 $\alpha = \text{yes}, \beta = \text{yes}$

→ ③ class NP Hard

→ Note: x is a decision problem.

→ x is NP Hard if every problem in NP is polynomial time reducible to x

$$Y \in P^*$$

$Y \rightarrow \text{all NP problems}$

→ x is hard as all problems in NP.

→ if x can be solved in polynomial time
then all problems in NP can be solved
in polynomial time.

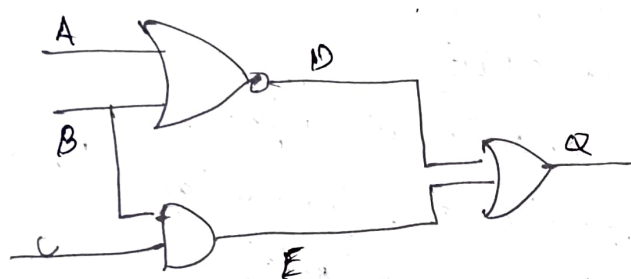
→ ④ NP complete

→ if the problem is NP as well as NP hard
then problem is NP complete.

eg:-

→ Not complete circuit sat problem

Note: Given a boolean circuit C is there an assignment to the variables that cause the circuit to output 1.



1 → yes
0 → no

eg:- SAT problem (Satisfiability)

Note: Given boolean expression ϕ is there an assignment to the variables that causes the expression to output 1.

$\phi = 1$ yes
 $\phi = 0$ no

$$\phi = (x_1 \wedge \neg x_2) \wedge (x_1 \vee x_2) \wedge (\neg x_1 \wedge x_2 \vee x_3)$$

eg:- 3 CNF SAT problem

① Literal: variables & its negation

ex:-

② Clause: OR of one or more literals $(x_1 \vee \neg x_2 \vee x_3)$

③ 3 CNF: AND of clauses $(x_1 \vee x_2) \wedge (\neg x_1 \vee x_2) \wedge (x_1 \vee \neg x_2)$
conjunctive normal form

④ 3 CNF: exactly 3 distinct literals in each clause.
ex: $(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3)$

→ 3-CNF SAT Given a 3-CNF expression ϕ , is there an assignment to the variables that causes the expression to output 1.

→ eg ④ clique problem:

→ Given graph $G(V, E)$ & an integer k , the problem is to find whether there is a subset V' of vertices of size at least k to determine if the graph contains a clique of size at least k .

→ eg:- 5 Vertex cover problem

→ Given graph $G(V, E)$ & +ve integer k , the problem is to find whether there is a subset V' of vertices of size at most k , such that every edge in the graph is connected to some vertex in V' .

→ NP complete proof

steps ① prove that the given problem is NP i.e. write a polynomial time verification algorithm.

② prove that the given problem is NP Hard.
i.e. write a polynomial time reduction algorithm from any NP problem to given problem.

possible in polynomial time

Circuit-SAT
CKT-SAT

↓
SAT

↓
3 CNF SAT

↓
Clique

↓
Vertex cover

↓
Subset sum

↓
HAMILTONIAN

↓
TSP

NP problem (Poly time only)
↓
given problem

(x_1, x_2, x_3)

or) prove that clique problem is NP complete.

→ ① write a polynomial time verification algorithm to prove that given problem is NP hard.

→ Input $\langle G, k, v' \rangle$

→ verify algorithm

→ ① Test whether v' is the set of k vertices in the graph G .

→ ② check whether for each pair $(u, v) \in v'$ the edge $(u, v) \in E$

→ ③ Both step pass then accept, otherwise Reject.

→ Alg will execute in polytime $\therefore \text{clique} \in \text{NP}$

→ ② write a polynomial time reduction algorithm from 3CNF Sat problem to clique problem.

→ Algorithm:-

$C_1, C_2, C_3, \dots, C_k \rightarrow \text{clauses}$

① Let $\phi = C_1 \wedge C_2 \wedge \dots \wedge C_k$ be a Boolean formula in 3CNF with k clauses. Each clause C_i has exactly 3 distinct literals l_1^i, l_2^i, l_3^i .

② construct a graph G ~~be~~ such that ϕ is satisfiable iff G has a clique of size k . The graph G is constructed as follows.

→ for each clause $C_i = (l_1^i \vee l_2^i \vee l_3^i)$ in ϕ , we place a tuple of vertices v_1^i, v_2^i, v_3^i into V .

Clause = k

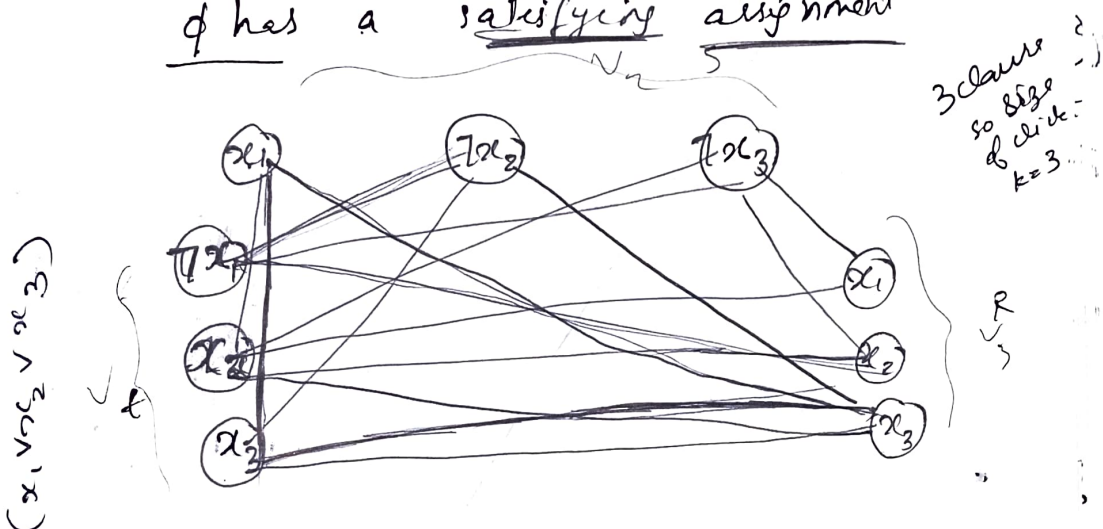
Clique size = k

→ put an edge b/w V_i^t to V_i^s if following 2 conditions hold.

→ V_i^t & V_i^s is different triples

→ l_i^r is not a negation of l_i^r

③ If G has a clique of size k , then ϕ has a satisfying assignment



3 clause
so size
of clique
 $k=3$

$$\phi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \neg x_3)$$

$$x_1 = 1$$

$$x_2 = 0$$

$$x_3 = 1$$

$$x_1 \vee \neg x_2 \vee x_3 = 1$$

$$\neg x_1 \vee x_2 \vee x_3 = 1$$

$$x_1 \vee x_2 \vee \neg x_3 = 1$$

Since not connect
one node & its
negation
no edges
 $x_1 \sim x_1$ (no edge)

we took
only 1
instance. $\phi \wedge \phi \wedge \phi$ $\wedge \wedge \wedge = 1$

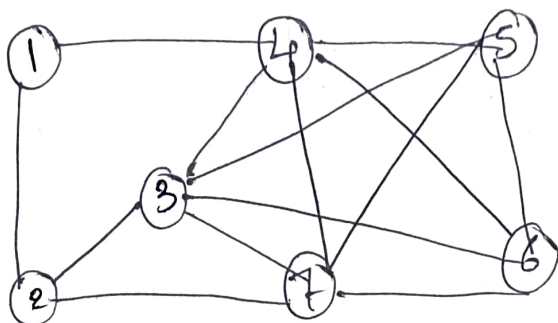
→ If G has a clique of size 3 then ϕ has a satisfying assignment at (x_1, x_2, x_3) clique

→ Reduction from 3-CNF SAT to clique is polynomial time: is possible

∴ clique belongs to NP Hard.
ie. by step 1 & 2 clique belongs to NP-complete.

to prove

vertex cover problem is np complete.



Polynomial time verification Algorithm $\langle G, k, V' \rangle$

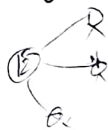
~~$\langle G, k, V' \rangle$~~

V = set of vertex
 G = Graph

$V' = V, V, V$

① count = 0

② for each vertex v in V' remove all edges adjacent to v from E
→ increment count by 1



③ if count is k & E is empty then solution is correct

④ else soln is wrong

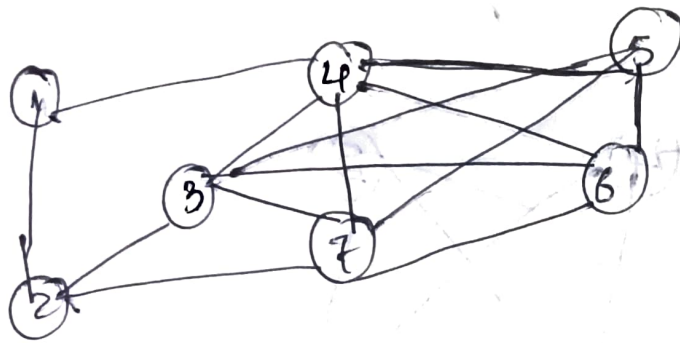
→ ∴ Vertex cover problem belongs to Class np

np hard
② Polynomial time reduction Algorithm ~~$\langle G, k \rangle$~~
 $\langle G = (V, E), k \rangle$

① construct G' which is complement of G

② if G' has a vertex cover $|V| - k$ then G has a clique of size k

→ ∴ Vertex cover belongs to np hard



already all from
3 is connected.
so we take 1

from 1 $\rightarrow$ 4x

1 $\rightarrow$ 2x

1 $\rightarrow$ 3x

1 $\rightarrow$ 5x

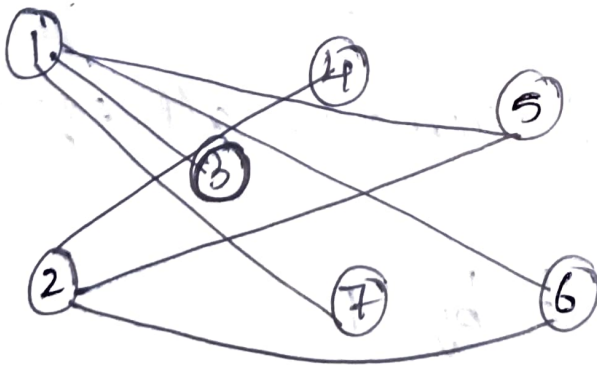
(need new con-
nection)

$$V \rightarrow 7$$

$$k \rightarrow 5 \quad (3, 4, 5, 6, 7)$$

$$|V| - k = |7| - 5 = 2 \rightarrow (\text{vertex cover size})$$

(i) G' graph complement (need new connections).



$\therefore$ vertex cover belong to np hard.

$\rightarrow$ vertex cover belong to np complete.
end module.